

## Semaine 1

### Fondamentaux du web & environnement

Anne Terrier — Développement web, classe passerelle

July 19, 2026

Cours 4h

TP 4h

- Comprendre le modèle client/serveur et le fonctionnement du web
- Distinguer les rôles de HTML, CSS et JavaScript
- Distinguer un site statique d'un site dynamique
- Installer et configurer son environnement de développement
- Créer son premier dépôt Git et effectuer un premier commit

## 1 Comment fonctionne le web ?

Quand tu tapes `https://www.hepia.ch` dans ton navigateur, une série d'échanges invisibles se produit en quelques millisecondes. Comprendre ce mécanisme est le point de départ de tout développeur web.

### Le modèle client / serveur

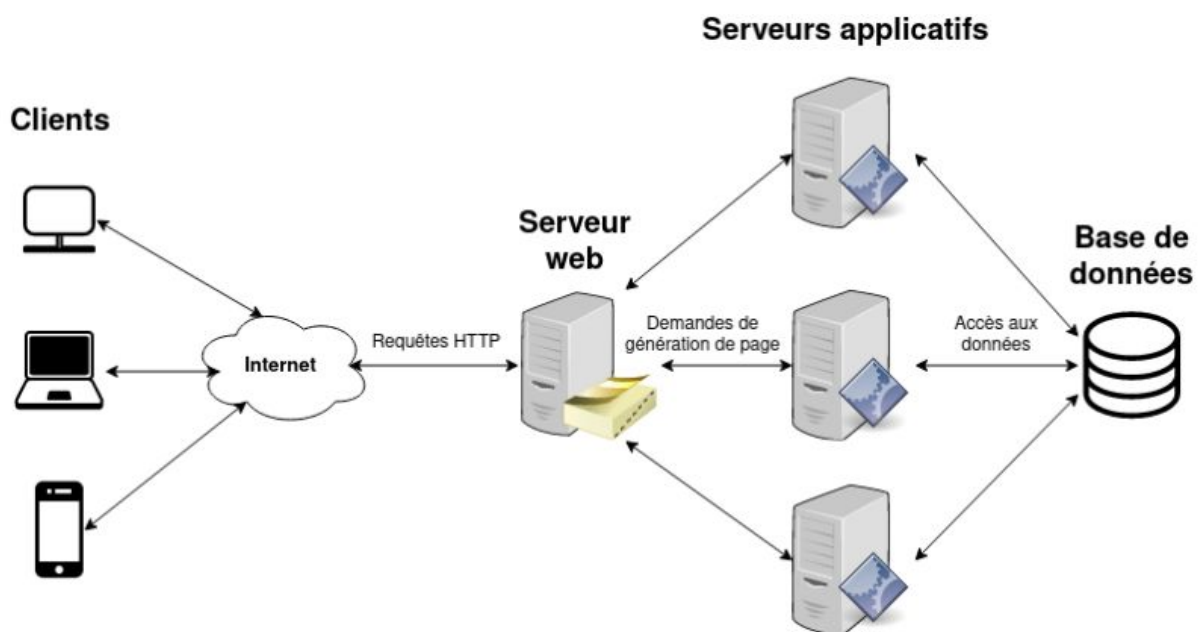
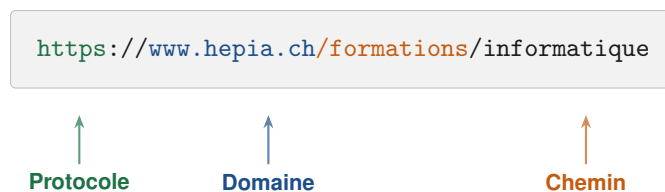


Figure 1: Architecteur Client-Serveur

**Client** : programme qui *demande* une ressource (le navigateur).

**Serveur** : programme qui *répond* à la demande en envoyant des fichiers ou des données. La communication se fait via le protocole **HTTP** (ou **HTTPS** pour la version sécurisée).

## Anatomie d'une URL



## DNS — de l'adresse au nom

Un ordinateur ne comprend pas `www.hepia.ch` : il a besoin d'une adresse numérique, appelée **adresse IP** (par exemple `192.0.2.34`). Le **DNS** (*Domain Name System*) est le service qui traduit un nom de domaine lisible en adresse IP.

Le DNS fonctionne comme un annuaire : tu connais le *nom* d'une personne (le domaine), l'annuaire te donne son *numéro* (l'adresse IP), et c'est ce numéro qui permet réellement d'établir la communication.

### Étapes simplifiées :

1. Le navigateur demande au DNS : « quelle est l'adresse IP de `www.hepia.ch` ? »
2. Le serveur DNS répond avec l'adresse IP correspondante
3. Le navigateur peut alors envoyer sa requête HTTP directement à cette adresse

## Les codes de statut HTTP

| Code | Signification | Quand ?                    |
|------|---------------|----------------------------|
| 200  | OK            | Tout s'est bien passé      |
| 301  | Redirection   | La page a changé d'adresse |
| 404  | Not Found     | La page n'existe pas       |
| 500  | Server Error  | Erreur côté serveur        |

## Sites statiques et sites dynamiques

Un **site statique** renvoie toujours le même contenu au client : le fichier HTML existe déjà sur le serveur et est envoyé tel quel.

Un **site dynamique** génère le contenu à la demande, souvent à partir d'une base de données : deux visiteurs, ou deux moments différents, peuvent recevoir des pages différentes.

|               | Site statique                  | Site dynamique                                 |
|---------------|--------------------------------|------------------------------------------------|
| Contenu       | Identique pour tout le monde   | Généré à chaque requête                        |
| Exemple       | Page vitrine simple, portfolio | Forum, boutique en ligne, réseau social        |
| Techniquement | Fichiers HTML/CSS/JS seuls     | Serveur applicatif (Node.js) + base de données |

### Où se situe MiniForum ?

Le projet fil rouge **MiniForum** commence comme un site **statique** (données codées en dur en semaine 1–5), puis devient **dynamique** à partir de la phase 3, lorsqu'il se connecte à une vraie base de données via Node.js/Express.

## 2 Les trois langages côté clients



*Les murs*

*La peinture*

*L'électricité*

Analogie : construction d'une maison

### Important — Qui exécute quoi ?

**HTML, CSS et JavaScript** s'exécutent dans le **navigateur** (côté client).

### Un premier exemple

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>MiniForum</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9   <h1>Bienvenue sur MiniForum</h1>
10  <button id="btn">Cliquez-moi</button>
11  <script src="app.js"></script>
12 </body>
13 </html>
```

Extrait 1: index.html — Structure

```
1 button {
2   background-color: #1D3557; /* bleu marine */
3   color: white;
4   padding: 10px 20px;
5   border: none;
6   border-radius: 4px;
7   cursor: pointer;
8 }
```

Extrait 2: style.css — Présentation

```
1 const btn = document.getElementById('btn');
2 btn.addEventListener('click', () => {
3   alert('Bonjour depuis JavaScript !');
4 });
```

Extrait 3: app.js — Comportement

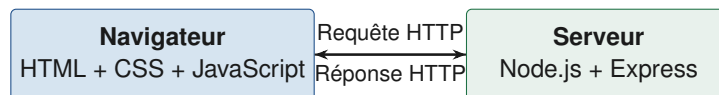
### 3 Les langages côté serveur

Jusqu'à présent, nous avons vu les langages exécutés dans le navigateur : **HTML**, **CSS** et **JavaScript**. Mais une application web doit souvent enregistrer et récupérer des données. Pour cela, le navigateur communique avec un programme exécuté sur un serveur.

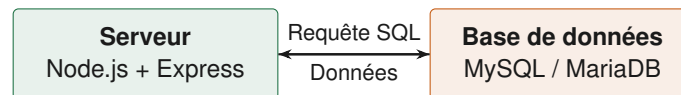
Un langage côté serveur est exécuté sur le serveur. Il reçoit une requête du navigateur, traite cette requête et renvoie une réponse.

#### Client et serveur

Le fonctionnement peut être résumé ainsi :



Le serveur peut également communiquer avec une base de données :



#### Quelques langages côté serveur

Il existe de nombreux langages permettant de développer des applications web.

| Langage                 | Exemple d'utilisation         |
|-------------------------|-------------------------------|
| PHP                     | Sites et applications web     |
| Python                  | Applications web et API       |
| Java                    | Applications d'entreprise     |
| C#                      | Applications web avec ASP.NET |
| JavaScript avec Node.js | Serveurs web et API           |

#### Node.js

Dans ce cours, nous utiliserons **Node.js**.

Node.js permet d'exécuter du JavaScript en dehors du navigateur.

Il permet notamment de :

- créer un serveur web ;
- recevoir des requêtes HTTP ;
- communiquer avec une base de données ;
- créer des API.

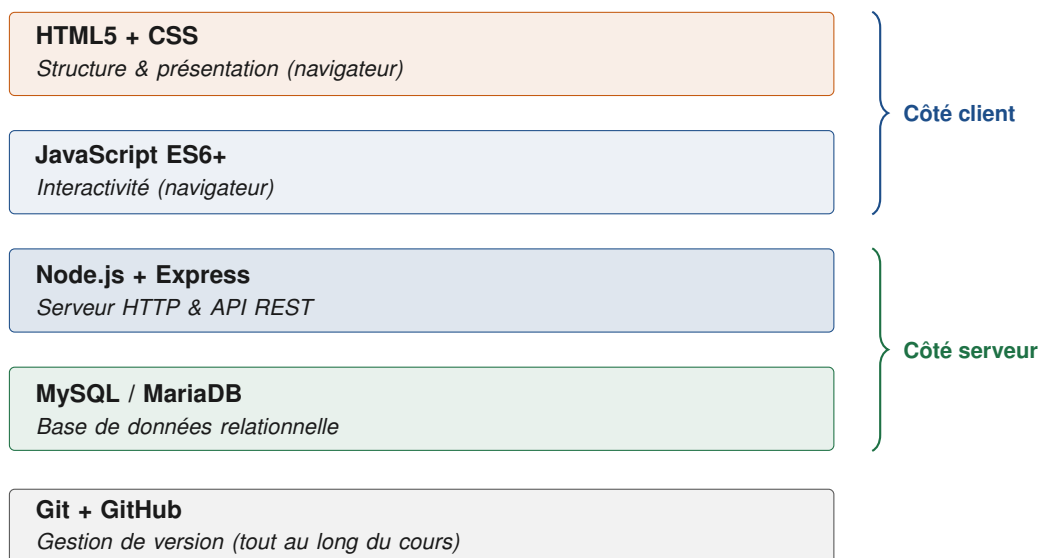
**Important — Côté client et côté serveur**

**JavaScript côté client** s'exécute dans le navigateur.

**JavaScript côté serveur** s'exécute sur le serveur avec Node.js.

## 4 La stack du cours

Dans notre projet **MiniForum**, les différentes technologies auront les rôles suivants :



Une application web complète est généralement composée de trois parties :

- une interface côté client ;
- un serveur côté serveur ;
- une base de données.

Dans ce cours, nous utiliserons JavaScript des deux côtés : **JavaScript dans le navigateur** et **Node.js sur le serveur**.

## 5 Environnement de développement

### Outils à installer

- |                      |                                                                   |
|----------------------|-------------------------------------------------------------------|
| • Visual Studio Code | <a href="https://code.visualstudio.com">code.visualstudio.com</a> |
| • Node.js LTS (v20+) | <a href="https://nodejs.org">nodejs.org</a>                       |
| • Git                | <a href="https://git-scm.com">git-scm.com</a>                     |
| • Compte GitHub      | <a href="https://github.com">github.com</a>                       |

## Extensions VS Code recommandées

| Extension   | Utilité                                    |
|-------------|--------------------------------------------|
| Prettier    | Formatage automatique du code              |
| ESLint      | Détection d'erreurs JavaScript             |
| Live Server | Rechargement automatique du navigateur     |
| REST Client | Tester les routes API depuis VS Code       |
| GitLens     | Visualiser l'historique Git dans l'éditeur |

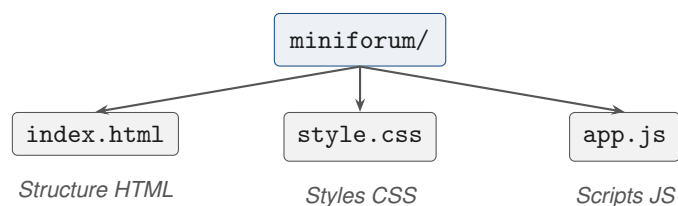
## Premiers pas dans le terminal

Git et Node.js s'utilisent en ligne de commande. Quelques commandes suffisent pour naviguer dans les dossiers de ton projet.

| Commande                              | Effet                               |
|---------------------------------------|-------------------------------------|
| <code>pwd</code>                      | Affiche le dossier courant          |
| <code>ls (ou <code>dir</code>)</code> | Liste le contenu du dossier courant |
| <code>cd nom-dossier</code>           | Se déplace dans un sous-dossier     |
| <code>cd ..</code>                    | Remonte d'un niveau                 |
| <code>mkdir nom</code>                | Crée un nouveau dossier             |
| <code>touch nom.txt</code>            | Crée un fichier vide (macOS/Linux)  |

Ces commandes seront revues et approfondies dans le cours Linux/Bash donné en parallèle. Il suffit ici de savoir se déplacer et créer un dossier pour démarrer le projet MiniForum.

## Structure du projet MiniForum



## 6 Git — Premiers pas

Git est un outil de **gestion de version** : il enregistre l'historique de chaque modification de ton code, permet de revenir en arrière en cas d'erreur, et facilite le travail en équipe. C'est un outil professionnel incontournable, utilisé dans tous les projets informatiques.

### Pourquoi utiliser Git ?

Imagine que tu travailles sur ton projet MiniForum. Sans Git, tu te retrouves rapidement dans cette situation :

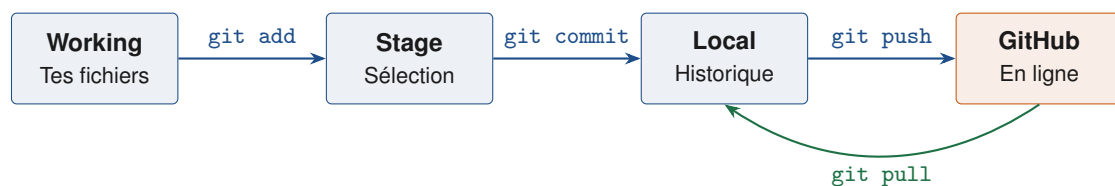
| Fichier               | Ce que tu essaies de faire |
|-----------------------|----------------------------|
| index.html            | Version qui marche         |
| index_final.html      | Tentative d'amélioration   |
| index_final2.html     | Autre tentative            |
| index_VRAI_final.html | On ne sait plus...         |

Git résout ce problème : un seul fichier, un historique complet, la possibilité de revenir à n'importe quel état passé.

- **Sauvegarder** l'historique complet de son projet, modification par modification
- **Revenir en arrière** à n'importe quel état antérieur du code
- **Travailler en équipe** sans écraser le travail des autres
- **Expérimenter** sans risque, sur une branche séparée
- **Synchroniser** son travail entre plusieurs machines via GitHub

### Les quatre zones de Git

Git organise le travail en quatre zones distinctes. Comprendre ces zones est essentiel pour ne pas être perdu face aux commandes.



| Zone                     | Description                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------------------|
| <b>Working Directory</b> | Tes fichiers tels que tu les vois dans ton éditeur. Toutes tes modifications vivent ici.        |
| <b>Stage (Index)</b>     | Zone de préparation : tu sélectionnes les fichiers que tu veux inclure dans le prochain commit. |
| <b>Repository local</b>  | L'historique Git stocké sur ta machine. Chaque commit est un instantané du projet.              |
| <b>Remote (GitHub)</b>   | Copie du dépôt hébergée en ligne. Permet la sauvegarde et le partage.                           |

Imagine que tu prépares une photo de groupe. **Working Directory** : tout le monde est dans la salle. **Stage** : tu demandes à certaines personnes de se mettre en position. **Commit** : tu prends la photo — c'est un instantané permanent. **Push** : tu envoies la photo sur un serveur partagé pour que tout le monde y accède.

## Commandes essentielles

```
1 # Initialiser un depot Git dans le dossier courant
2 git init
3
4 # Voir l'etat des fichiers (modifies, en attente, non suivis)
5 git status
6
7 # Ajouter tous les fichiers modifies au stage
8 git add .
9
10 # Ajouter un seul fichier specifique
11 git add index.html
12
13 # Enregistrer un commit avec un message
14 git commit -m "Premier commit : structure de base du MiniForum"
15
16 # Voir l'historique des commits
17 git log --oneline
```

### Extrait 4: Initialiser un projet Git

```
1 # Lier le depot local a un depot GitHub
2 git remote add origin https://github.com/votre-login/miniforum.git
3
4 # Envoyer les commits sur GitHub (premiere fois)
5 git push -u origin main
6
7 # Envoyer les commits suivants
8 git push
9
10 # Recuperer les modifications depuis GitHub
11 git pull
```

### Extrait 5: Synchroniser avec GitHub

```
1 # Voir quelles modifications ont ete faites sur un fichier
2 git diff index.html
3
4 # Annuler les modifications d'un fichier (avant git add)
5 git restore index.html
6
7 # Voir le detail d'un commit
8 git show abc1234
```

### Extrait 6: Revenir en arriere



Un bon message de commit décrit **ce qui a changé**, pas comment.

**OK** Ajout formulaire création de sujet

**OK** Correction bug affichage liste des messages

**NON** modif      **NON** ca marche      **NON** update

## Le fichier .gitignore

Certains fichiers ne doivent jamais être versionnés : mots de passe, dépendances téléchargées, fichiers temporaires. Le fichier .gitignore indique à Git quels fichiers ignorer.

```
1 # Dépendances Node.js (à ne jamais committer)
2 node_modules/
3
4 # Variables d'environnement (mots de passe, clés API)
5 .env
6
7 # Fichiers système macOS
8 .DS_Store
9
10 # Fichiers temporaires de l'éditeur
11 .vscode/settings.json
```

Extrait 7: .gitignore — Exemple pour MiniForum

### Important — Ne jamais committer node\_modules/

Le dossier node\_modules/ peut contenir des milliers de fichiers et peser plusieurs centaines de mégaoctets. Il doit **toujours** figurer dans .gitignore : les dépendances se réinstallent en une commande (npm install), elles n'ont pas leur place dans l'historique Git.

## 7 Lexique de la semaine

| Terme              | Définition                                                                    |
|--------------------|-------------------------------------------------------------------------------|
| URL                | Adresse permettant de localiser une ressource sur le web                      |
| DNS                | Service qui traduit un nom de domaine en adresse IP                           |
| HTTP / HTTPS       | Protocole de communication entre client et serveur (HTTPS = version chiffrée) |
| Client             | Programme qui demande une ressource (le navigateur)                           |
| Serveur            | Programme qui répond à une demande en fournissant fichiers ou données         |
| Statique           | Contenu identique quel que soit le visiteur ou le moment                      |
| Dynamique          | Contenu généré à la demande, souvent depuis une base de données               |
| Commit             | Enregistrement d'un instantané des modifications dans Git                     |
| Dépôt (repository) | Dossier suivi par Git, contenant l'historique du projet                       |

## Pour aller plus loin

- Un **quiz d'auto-évaluation** sur cette semaine est disponible sur Moodle.
- La **fiche TP** (mise en place de l'environnement et premier dépôt Git) est distribuée séparément : *S01\_TP.pdf*.

## Résumé de la semaine

- Le web fonctionne sur un modèle **client / serveur** via le protocole **HTTP**
- Le **DNS** traduit un nom de domaine en adresse IP
- **HTML** structure, **CSS** présente, **JavaScript** rend interactif
- Un site **statique** renvoie toujours le même contenu, un site **dynamique** le génère à la demande
- **Git** : add → commit → push pour versionner son code
- **MiniForum** sera le projet fil rouge développé progressivement sur 15 semaines

## Pour la semaine 2

- Vérifier que VS Code, Node.js et Git sont correctement installés
- Créer son compte GitHub si ce n'est pas déjà fait
- Avoir son dépôt miniforum/ initialisé avec les trois fichiers de base et poussé sur GitHub